



endingo

정훈이의 공부일지

CATEGORY



👋 안녕하세요, AI 개발자 이정훈입니다

AI · 백엔드 · 데이터 분석 중심의 프로젝트와 경험을 공유합니다.

 Contact Me

Data science/딥러닝

[딥러닝] 4. 이진분류 모델링

endingo 2025. 4. 7. 19:15

이진분류 모델링은
결과 값이 두 개 중에 하나로 분류된다는 것이다.

★ 이진분류 모델링 활성화함수: 시그모이드 함수

★ 이진분류 목적함수: BCE

1. 데이터 준비

1) 데이터 준비

x: 'Sex', 'Age', 'Fare' 라는 특성

y: 'Survived' 라는 정답 레이블

```
data.loc[:, features]
```

```
data.loc[:, target]
```

```
target = 'Survived'
```

```
features = ['Sex', 'Age', 'Fare']
```



정훈이의 공부일지



2) 가변수화

성별은 범주값이니까 가변수화해줘야된다.

```
pd.get_dummies(drop_first=True)
```

성별만 범주화니까 가변수화 해줘야됨 'True'가 1 'False'가 0 수치화된것

```
x = pd.get_dummies(x, columns = ['Sex'], drop_first = True)
```

```
x.head()
```

	Age	Fare	Sex_male
0	22.0	7.2500	True
1	38.0	71.2833	False
2	26.0	7.9250	False
3	35.0	53.1000	False
4	35.0	8.0500	True

그러면 이런식으로 모두 딥러닝 모델이 이해하게 수치형으로 바꾸어주었다.

3) 데이터분할

```
train_test_split():
```

훈련: x_train, y_train

검증: x_val, y_val

```
x_train, x_val, y_train, y_val = train_test_split(x, y, test_size=.3, random_state = 20)
```

4) 스케일링

정규화하여서 범위를 똑같이해줘야지 기계가 오해 안 한다.

```
scaler = MinMaxScaler()
```

```
x_train = scaler.fit_transform(x_train)
```

```
x_val = scaler.transform(x_val)
```

2. 모델링

1) 딥러닝을 위한 준비작업

make_DataSet: x_train, y_train, x_val, y_val을 통해 모델에 데이터를 로더한다.

make_DataSet 함수는 전처리를 끝마친 데이터를 텐서로 변환하고 이를 텐서 데이터셋으로 합쳐서 데이터로더를 통해 딥러닝에 데이터를 전달하는 기능을 한다.



정훈이의 공부일지



첫번째 배치만 로딩해서 살펴보기

```
for x, y in train_loader:
    print(f"Shape of x [rows, columns]: {x.shape}")
    print(f"Shape of y: {y.shape} {y.dtype}")
    break
```

```
Shape of x [rows, columns]: torch.Size([32, 3])
Shape of y: torch.Size([32, 1]) torch.float32
```

배치크기 32, 특성수 3의 텐서로 잘 전달되었다.

2) 모델선언

```
n_feature = x.shape[1]
```

모델 구조 설계

```
model = nn.Sequential(
    nn.Linear(n_feature, 1),
    nn.Sigmoid()      # [회귀와 다른점] 이진분류 모델링이니까: 출력층 시그모이드 활성화 함수 추가
).to(device)
```

```
print(model)
```

```
Sequential(
  (0): Linear(in_features=3, out_features=1, bias=True)
  (1): Sigmoid()
)
```

이런식으로 신경망 하나(입력이 3이고 출력이 1인)가 생성되고
활성함수는 이진분류니까 시그모이드 함수를 쓴다.

3) 목적함수, 손실함수, 옵티마이저 선언

- 이진분류에서는 목적함수를 BCE로 쓴다.

```
loss_fn = nn.BCELoss()      # 회귀는 MSE, 이진분류는 BCE[회귀와 다른점] Binary Cross Entropy
optimizer = Adam(model.parameters(), lr=0.01) # Adam은 똑같아
```

4) 모델학습

```
epochs = 50
tr_loss_list, val_loss_list = [], []
```



정훈이의 공부일지



리스트에 loss 추가 --> learning curve 그리기 위해.

```
tr_loss_list.append(tr_loss)
```

```
val_loss_list.append(val_loss)
```

```
print(f"Epoch {t+1}, train loss : {tr_loss:4f}, val loss : {val_loss:4f}")
```

앞서 선언했던 train, evaluate 함수를 사용하여

epochs수, loss_fn, optimizer, model 등의 매개변수를 확인하고 훈련을 시켜보자

```
Epoch 45, train loss : 0.496314, val loss : 0.504169
Epoch 46, train loss : 0.498135, val loss : 0.503745
Epoch 47, train loss : 0.492840, val loss : 0.504295
Epoch 48, train loss : 0.500504, val loss : 0.504862
Epoch 49, train loss : 0.492714, val loss : 0.504642
Epoch 50, train loss : 0.494401, val loss : 0.504668
```

- 훈련이 정상적으로 진행되었다면 학습된 파라미터 확인, 학습곡선을 확인해보자

파라미터는 훈련할 때 마다 다르며 학습이 진행되면서 조정되게 된다.

$f = -0.2814 \cdot A + 4.4883 \cdot F - 2.3769 \cdot S + 0.8241$ 이런식으로 시그모이드 함수 x값에 들어가게 된다.

```
for name, param in model.named_parameters():
```

```
    if param.requires_grad:
```

```
        print(f"Parameter: {name}, Value: {param.data}")
```

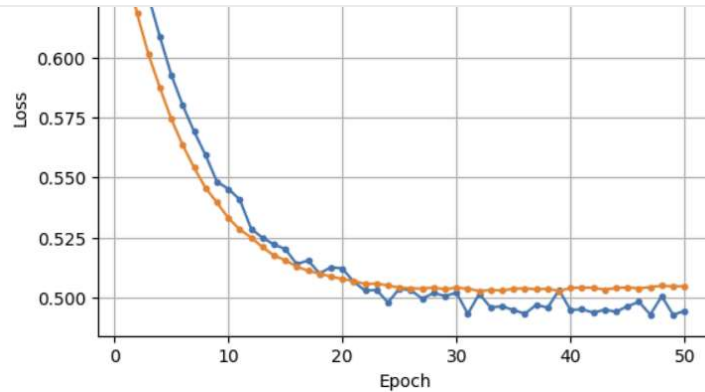
```
Parameter: 0.weight, Value: tensor([[ -0.2814,  4.4883, -2.3769]])
Parameter: 0.bias, Value: tensor([0.8241])
```

학습 곡선을 봤을 때 val_loss, train_loss가 일정하게 감소하고있는 것을 확인할 수 있으며 잘 진행되고있다고 육안적으로 확인할 수 있다.

```
dl_learning_curve(tr_loss_list, val_loss_list)
```



정훈이의 공부일지



5) 모델 평가

```
_, pred = evaluate(x_val_ts, y_val_ts, model, loss_fn, device)
```

마지막으로 evaluate 함수를 사용하여 pred에 평가점수를 저장한다. _는 변수를 사용하지않겠다. 즉 사용하지않겠다고 선언한 것이다.

- 회귀와 다른점은 이진분류는 예측 결과가 0 ~ 1사이의 확률값이라는 점이다.

```
pred.numpy()[:5]
```

```
array([[0.6984064 ],
       [0.15899362],
       [0.20099726],
       [0.17513818],
       [0.69917023]], dtype=float32)
```

- 예측 결과가 0 ~ 1사이의 확률값으로 나왔으면 이것은 '무엇무엇이다~' 라는 결론이 나와야되는데 이를 위해서 해야되는게 분류이다. 이 작업이 가능하려면 np.where() 함수를 써야한다.

예를들어 0.5 보다 예측확률이 크면 1(강아지) 이고 아니면 0(고양이)이다.

```
# 외우기
```

```
pred = np.where(pred.numpy() > .5, 1, 0)
pred[:5]
```

```
array([[1],
       [0],
       [0],
       [0],
       [1]])
```

결과)

강아지

고양이

고양이



정훈이의 공부일지



- 혼동행렬로 나타내기

```
confusion_matrix(y_val_ts.numpy(), pred)
```

```
array([[141, 29],
       [ 31, 67]])
```

0으로 예측했는데 0인: 141

0으로 예측했는데 1인: 31

1로 예측했는데 1인: 67

1로 예측했는데 0인: 29

- 혼동행렬을 통해 정보 뽑아내기(정확도, 정밀도, 재현률, f1-score)

```
print(classification_report(y_val_ts.numpy(), pred))
```

	precision	recall	f1-score	support
0.0	0.82	0.83	0.82	170
1.0	0.70	0.68	0.69	98
accuracy			0.78	268
macro avg	0.76	0.76	0.76	268
weighted avg	0.78	0.78	0.78	268

3. 전체 feature 사용

나머지 과정은 똑같고 데이터 전처리 부분에서만 다르다

특성 수를 많이해서 딥러닝 모델 선언

1) 데이터준비

x를 레이블 뺀 전체를 쓴다.

```
target = 'Survived'
x = data.drop(target, axis = 1)
y = data.loc[:, target]
```

2) 가변수화

Pclass, Sex, Embarked만 범주형이어서 drop_first=True를 해준다.

그럼 수치형으로 변

```
cat_cols = ['Pclass', 'Sex', 'Embarked']
x = pd.get_dummies(x, columns = cat_cols, drop_first = True)
```



정훈이의 공부일지



False	True	False	False	True
False	False	False	False	True
False	True	True	False	True
...	...	...	...	...
True	False	True	False	True
False	False	False	False	True
False	True	False	False	True
False	False	True	False	False
False	True	True	True	False

그럼 이런식으로 수치형으로 표현된다.

3) 모델 설계

위 처럼 데이터 특성수를 조절하면 더 나은 성능을 기대 할 수 있다.

```
n_feature = x.shape[1]
```

모델 구조 설계

```
model = nn.Sequential(
    nn.Linear(n_feature, 1),
    nn.Sigmoid()
    # 시그모이드 활성화 함수 추가
).to(device)
```

```
print(model)
```

```
Sequential(
  (0): Linear(in_features=8, out_features=1, bias=True)
  (1): Sigmoid()
)
```

4. 은닉층

1) 모델 선언

출력값의 활성화함수는 시그모이드이지만

은닉층의 활성화함수는 성능이 가장 좋은 ReLU()를 쓰도록하자

```
n_feature = x.shape[1]
```

모델 구조 설계

```
model = nn.Sequential(
    nn.Linear(n_feature, 5), # 은닉층
    nn.ReLU(),              # 은닉층의 활성화함수
    nn.Linear(5, 1),        # 출력층
    nn.Sigmoid()            # 출력층의 활성화함수
).to(device)
```



정훈이의 공부일지



공감

구독하기

Data science > 딥러닝' 카테고리의 다른 글

[딥러닝] 6. 다중 클래스 분류 모델링 - Mnist (0)

[딥러닝] 5. 다중 클래스 분류 모델링 - Iris (0)

[딥러닝] 3. 회귀 모델링 (0)

[딥러닝] 2. Pytorch (0)

[딥러닝] 1. 딥러닝 개요 (0)

정훈이의 공부일지

개발하면서 겪는 시행착오와 배움을 기록합니다. 백엔드, AI, 데이터 분석, 프로젝트 경험 중심으로 운영 중입니다. 공부하면서 얻은 인사이트를 함께 나누고 싶어요!

구독하기

댓글 0

이름

비밀번호

로그인 댓글만 허용한 블로그입니다

등록

공지사항

최근에 올라온 글

최근에 달린 댓글

Total

188

[딥러닝] 6. 다중 클래스 분류 모...

[딥러닝] 5. 다중 클래스 분류 모...

[딥러닝] 4. 이진분류 모델링

Today

9

Yesterday

114



정훈이의 공부일지



github git

일	월	화	수	목	금	토	
		1	2	3	4	5	2025/04 (16)
6	7	8	9	10	11	12	2025/03 (7)
13	14	15	16	17	18	19	
20	21	22	23	24	25	26	
27	28	29	30				

Blog is powered by Tistory / Designed by Tistory

 이정훈 개발자

AI와 백엔드, 데이터 분석을 좋아하는 개발자입니다.
다양한 문제 해결과 창의적인 개발을 추구합니다.

 이메일 보내기